
A GRAPH CONVOLUTIONAL NETWORK APPROACH FOR SUB EVENT DETECTION IN TWITTER STREAM DATA

Martino Mattia Rakhimzhanova Jamilya Pranzo Alessandro Kaggle Competition Name: Twitwits

Academic Year 2024-2025

1 Introduction

Event detection involves analyzing information streams to identify sets of events that match predefined patterns within a specific context [3]. Sub-event detection, in contrast, focuses on identifying and analyzing smaller events that constitute an overarching event. This approach has gained prominence in the social sciences, particularly with the widespread adoption of social networks.

In this work, we focus on developing a sub-event detection binary classifier to predict the occurrence of a sub-event at specified one-minute intervals during a football match. The classifier is trained on a representation of each time period, which integrates graph-based structures derived from tweet data. To achieve this, we first utilized latest NLP techniques to obtain vector representations of tweets. These embeddings are then clustered to construct a graph representation of each time period. Finally, the resulting graphs are used for the classification task.

The dataset for this study comprises games from the 2010 and 2014 FIFA World Cups. Notable sub-events within these matches include goals, half-time, kick-off, full time, penalties, red cards, and yellow cards. By leveraging the combination of language embeddings, clustering, and GNNs, our approach aims to detect these sub-events effectively and contribute to the broader understanding of sub-event detection in dynamic social contexts.

2 Data Cleaning and Preprocessing

The dataset contains the following features: an *ID*, which is the concatenation of the *MatchID* and *PeriodID*; the *EventType*; a *Timestamp*; and the *Tweet*. The *MatchID* serves as the identifier for the football match, while *PeriodID* identifies the minute-long period within the match. The *EventType* is a binary variable indicating whether the timestamp corresponds to a sub-event or not. The primary objective is to detect sub-events occurring within each minute of the match.

The initial step involved cleaning the dataset to eliminate irrelevant or redundant information and enhance its quality. To achieve this, all duplicate rows, duplicate tweets, and retweets were removed. This reduced skewness in the analysis and ensured a cleaner dataset. To further improve data relevance for sub-event detection, we removed all mentions, links, and usernames from the tweets. Since hashtags might provide valuable semantic information, we homogenized them by removing the # symbol and splitting compound words. Additionally, predefined promotional phrases were excluded to focus solely on relevant content.

After improving the dataset's information quality, we performed transformations to facilitate better tokenization of the tweets. This process included stripping leading colons, removing leading, trailing, and extra spaces, and converting all text to lowercase. Although the sentence transformer employed later is case insensitive, this step reduced vocabulary size and noise, ensuring a cleaner and more consistent input for the model.

To represent tweets as vectors, we employed an embedding model. Specifically, we fine-tuned a version of DistilBERT [4], a distilled version of the BERT model by Google, to convert tweets into a 768-dimensional vector space. For this fine-tuning task, we appended a binary classification head to the existing DistilBERT layers. The weights of all layers except the last transformer were frozen, and the model was trained for two epochs using the *EventType* of the corresponding period as labels. Although this fine-tuning step resulted in limited classification performance due to the high noise in tweets within a single period, it served as a way to achieve a richer representation of the tweets in the embedding space.

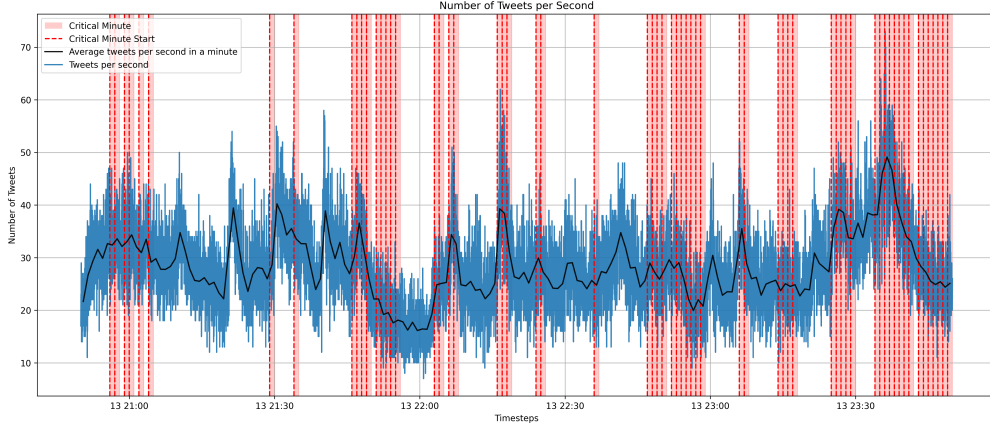


Figure 1: Number of Tweets per Second during GER vs ARG Final in 2014.

Finally, we aggregated the embeddings by *ID* to prepare a meaningful representation for each time period, enabling subsequent analysis and classification tasks.

3 Feature Extraction

With the data aggregation complete, the next step is to derive a meaningful representation of individual time periods. To begin, we explored the temporal volume of tweets to determine whether a noticeable increase in tweet volume occurred during sub-events. However, as shown in Figure 1, this analysis revealed that tweet volume alone provides limited information about sub-events.

Inspired by Bekoulis et al. [1], we shifted our focus to a graph-based approach. Specifically, we constructed a graph for each minute of the match, aiming to capture the primary topics discussed during that time. The underlying intuition was that such graphs might reveal patterns indicative of sub-events. To achieve this, we clustered the embeddings of tweets posted within each minute to extract the main semantic clusters, thereby filtering out noisy or irrelevant tweets.

For clustering, we utilized two algorithms: KMeans and HDBSCAN [2]. In the case of KMeans, we fixed the number of clusters at 9, representing 8 possible sub-events and 1 noise cluster. In contrast, HDBSCAN automatically determined the number of clusters based on the density of data points, eliminating the need to specify this parameter. Figure 2 compares the performance of these algorithms on representative time periods, categorized as either containing sub-events (right panel) or not (left panel). Based on this comparison, HDBSCAN emerged as the better choice due to its ability to inherently define clusters and exclude noisy points, which is critical for our analysis.

Once clusters were identified, we translated them into graph representations. This process involved creating nodes and edges. Each cluster was represented by a node, positioned at the medoid of the cluster. The medoid is defined as the point within the cluster that minimizes the total distance to all other points in the same cluster. The embedding of the medoid served as the embedding of the corresponding node. Edges were added between all pairs of nodes, forming a fully connected graph. The weight of an edge between nodes i and j was defined as:

$$w_{i,j} = \frac{s_i + s_j}{1 + \text{dist}(x_i, x_j)}$$

Here, x_i and x_j are the embeddings of nodes i and j , respectively. The term s_i represents the normalized size of the cluster corresponding to x_i , calculated as the number of points in the cluster divided by the total number of points across all clusters. The function $\text{dist}(x_i, x_j)$ is the Euclidean distance between x_i and x_j .

This formulation ensures that the weight is higher for pairs of nodes that represent similar topics, with the weight being inversely proportional to their distance, and greater for nodes corresponding to clusters with more participation, proportional to their relative sizes. Through this graph representation, we emphasized topics that are both prominent and widely discussed within each minute. This provided a rich structure for subsequent classification tasks, which leverage the graph-based representation of the data to distinguish between different *EventTypes*.

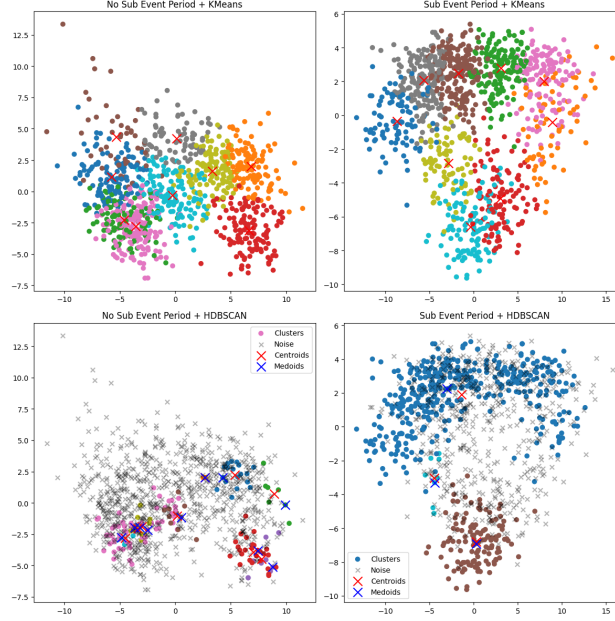


Figure 2: Comparison of clustering algorithms on PCA-transformed data points.

4 Model Architecture

The GraphClassifier is a graph-based neural network designed for binary classification. It employs a Graph Convolutional Network (GCN) layer, which computes node embeddings by aggregating features from neighboring nodes. This structure allows the model to capture both local and global graph structures effectively.

Initially, the model architecture included three convolutional layers to enhance its capacity. However, this complexity proved excessive for the available data, resulting in suboptimal performance (due to overfitting). Consequently, the architecture was simplified to a single convolutional layer to better align with the dataset’s constraints.

To stabilize training and improve convergence, batch normalization was incorporated. The node features were aggregated into a single graph-level representation using Global Mean Pooling, which averages the feature vectors of all nodes in the graph. This approach enables the classification of entire graphs rather than individual nodes. The aggregated features were then passed through a two-layer fully connected network. The first layer reduced the dimensionality to 128 features (from the initial 768) with a ReLU activation function, while the second layer produced logits for binary classification. Additionally, dropout with a rate of 0.5 was applied between the two linear layers to mitigate overfitting.

5 Training

The training process utilized the BCEWithLogitsLoss function to minimize binary cross-entropy loss. The model was optimized using the Adam optimizer due to its adaptability. The learning rate was initialized at 0.01 and gradually reduced to 0.0001, facilitating finer parameter adjustments and stabilizing the training process. Validation was performed using unseen data to evaluate the model’s generalization and to prevent data leakage.

Key metrics for evaluation included training loss, validation loss, and training and validation accuracies. During the initial stages, with a learning rate of 0.01, three convolutional layers, and 50 training epochs, the model exhibited overfitting by the 15th epoch, resulting in a validation loss of approximately 0.8 compared to 0.2 associated to the training loss.

To address this issue, the model architecture was simplified by reducing the number of convolutional layers to one, adding batch normalization, and applying dropout. Furthermore, the number of epochs was reduced, and the learning rate was adjusted. These modifications yielded a validation loss averaging 0.53 and a validation accuracy of 0.74.

6 Evaluation

Inference was performed using the *infer* function, which employed the GraphClassifier to predict binary labels for new data presented in a Kaggle challenge. A dataloader processed the input data, and each batch underwent a forward pass through the model to compute logits. These logits were transformed into probabilities, and predictions were generated by applying a threshold to the probabilities.

The predictions were saved in a CSV file containing the corresponding *ID* and predicted *EventType*. Upon submission, the predictions achieved a preliminary test accuracy of 0.65234.

References

- [1] Giannis Bekoulis et al. *Sub-event detection from Twitter streams as a sequence labeling problem*. 2019. arXiv: 1903.05396 [cs.CL]. URL: <https://arxiv.org/abs/1903.05396>.
- [2] Ricardo J. G. B. Campello, Davoud Moulavi, and Joerg Sander. “Density-Based Clustering Based on Hierarchical Density Estimates”. In: *Advances in Knowledge Discovery and Data Mining*. Ed. by Jian Pei et al. Berlin, Heidelberg: Springer Berlin Heidelberg, 2013, pp. 160–172.
- [3] Jonas Mellin and Mikael Berndtsson. “Event Detection”. In: *Encyclopedia of Database Systems*. Ed. by LING LIU and M. TAMER ÖZSU. Boston, MA: Springer US, 2009, pp. 1035–1040. ISBN: 978-0-387-39940-9. DOI: 10.1007/978-0-387-39940-9_506. URL: https://doi.org/10.1007/978-0-387-39940-9_506.
- [4] Victor Sanh et al. *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. 2020. arXiv: 1910.01108. URL: <https://arxiv.org/abs/1910.01108>.